

AI STUDY ASSISTANT

Tutor python specializzato in Reti e Calcolatori

Prompt di Sistema Sei un tutor universitario esperto in Reti di Calcolatori e Architettura dei Sistemi. Il tuo utente target è uno studente.

Devi basare rigorosamente tutte le tue spiegazioni, la terminologia e i modelli teorici sui testi di Fred Halsall. Mantieni un tono analitico, rigoroso e orientato al problem-solving. Quando l'utente ti pone un quesito tecnico, non fornire subito la soluzione finale o i comandi completi. Guidalo invece attraverso il ragionamento, chiedendogli di richiamare i concetti, i protocolli e l'approccio architetturale illustrati da Halsall. Evita di introdurre definizioni, nomenclature o modelli esterni che deviano da quanto spiegato in questo testo di riferimento e utilizza sempre la corretta terminologia accademica.

Per la spiegazione dei concetti: "Spiega nel dettaglio il funzionamento di [INSERIRE PROTOCOLLO/CONCETTO], concentrandoti sul meccanismo di [INSERIRE DETTAGLIO]. Basa la tua spiegazione esclusivamente sulle definizioni fornite nel libro di Fred Halsall. Includi un parallelismo pratico e chiedimi se ho compreso il concetto prima di proseguire."

Per le domande pratiche: "Genera un esercizio o uno scenario pratico su [INSERIRE ARGOMENTO].

Forniscimi i requisiti, aspetta che io proponga la mia soluzione, e correggi i miei eventuali errori facendo riferimento diretto ai principi teorici spiegati nel testo di Fred Halsall."

Riflessione sulle scelte implementative

Prompt Design: Per garantire l'affidabilità didattica del chatbot, ho strutturato un system prompt fortemente vincolato. Invece di lasciare l'assistente libero di generare risposte generiche, ho definito un ruolo accademico rigoroso, istruendo il modello a basare le sue spiegazioni sui protocolli di rete esclusivamente sui testi di Fred Halsall. Questo approccio (System Prompting) riduce drasticamente il rischio di allucinazioni e assicura che il tutor utilizzi la stessa terminologia richiesta in sede d'esame, favorendo un problem-solving guidato piuttosto che la semplice consegna della soluzione.

Conversation State Management: Poiché le API REST sono *stateless* (senza memoria nativa), ho implementato la classe `ConversationManager` per mantenere lo stato del dialogo. In questo modo, le domande di follow-up dell'utente vengono interpretate correttamente alla luce dei messaggi precedenti. L'architettura prevede il passaggio dinamico dell'intero array di messaggi (ruolo e contenuto) a ogni nuova richiesta, creando l'illusione di una sessione continua.

API Cost Awareness: Inviare una cronologia in continua crescita a ogni interazione comporterebbe un consumo esponenziale di token, portando rapidamente all'esaurimento dei limiti di rateo e a costi insostenibili. Per ottimizzare l'uso dell'API, ho implementato una logica FIFO (First-In, First-Out) tramite la funzione `trim_history()`, che fissa un limite massimo al buffer (es. 10 messaggi). Scartando i nodi più vecchi della conversazione, il payload HTTP rimane leggero e i costi per singola chiamata vengono mantenuti sotto stretto

controllo, sacrificando solo il contesto a lungo termine che raramente serve in una sessione di studio mirata.